

Веб-приложение в Kubernetes с Gateway API, метриками и логами

Автор Схрейдер

Репозиторий github.com/krantro2938/mts-devops-k8s

Ветка main

1 Архитектура и состав решения

ВЕРСИЯ KUBERNETES

v1.36.5

containerd 2.2.9 · runc 1.4.3 · CNI Calico 3.33

РЕАЛИЗАЦИЯ GATEWAY API

Envoy Gateway v1.9.2

Gateway API v1.6.1, standard channel

ИНСТРУМЕНТ ЛОГИРОВАНИЯ

Fluentd v1.19

DaemonSet → хранилище VictoriaLogs v1.53

ОС, НА КОТОРОЙ ТЕСТИРОВАЛОСЬ

Ubuntu 24.04 LTS

чистая VM и CI-раннер на каждый коммит

СПОСОБ РАЗВЁРТЫВАНИЯ KUBERNETES

kubeadm

конфиг v1beta4, скрипт `bootstrap-node.sh`

ИНСТРУМЕНТЫ АВТОМАТИЗАЦИИ

Bash · Helm 4 · Kustomize · Make

CI в GitHub Actions; версии закреплены в `versions.env`

СПОСОБ РАЗВЁРТЫВАНИЯ PROMETHEUS

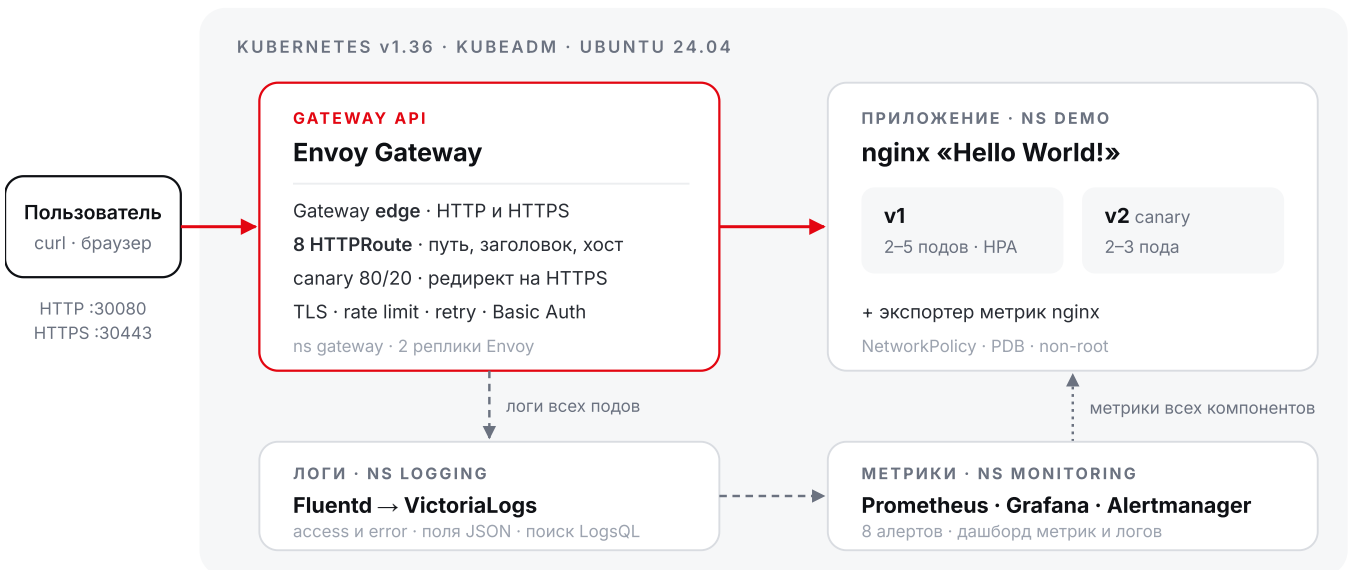
Helm: kube-prometheus-stack

Prometheus Operator, Grafana, Alertmanager

ПРИЛОЖЕНИЕ

nginx 1.30 — «Hello World!»

версии v1 и v2, JSON access-лог, экспортер метрик



Сплошная линия — HTTP-трафик, пунктир — логи, точки — сбор метрик. Интерфейсы Grafana, Prometheus и VictoriaLogs опубликованы через тот же Gateway по имени хоста и закрыты паролем.

make up

одна команда от чистой Ubuntu до проверки

12 мин

полное развёртывание на чистой VM

42

автоматические проверки всех требований

2x

повторный запуск безопасен, проверено в CI

2 Реализованный функционал — обязательные пункты

Всё проверяется автоматически: `./scripts/smoke-test.sh`. Ниже — ручная проверка, `IP` — адрес узла.

ПУНКТ	КАК РЕАЛИЗОВАНО	ПОЧЕМУ ТАК	КАК ПРОВЕРИТЬ
01 Kubernetes	<code>kubeadm init</code> с конфигом <code>v1beta4</code> ; <code>containerd</code> и <code>runc</code> с проверкой <code>sha256</code> ; Calico; <code>StorageClass local-path</code> ; <code>metrics-server</code> .	<code>kubeadm</code> — приоритет по условию, без привязки к вендору. Calico поддерживает <code>NetworkPolicy</code> .	<code>kubectl get nodes -o wide</code> → Ready, v1.36.5
02 Веб-приложение	Свой Helm-чарт: <code>nginx-unprivileged</code> , конфиг в <code>ConfigMap</code> , версии <code>v1</code> и <code>v2</code> , <code>HPA</code> , <code>PDB</code> , пробы. <code>Access-лог</code> в <code>JSON</code> , <code>error-лог</code> отдельно.	Код не нужен — достаточно конфигурации. <code>JSON-лог</code> сразу раскладывается на поля.	<code>curl http://IP:30080/</code> → Hello World!
03 Gateway API	<code>GatewayClass envoy</code> → <code>Gateway edge</code> (<code>HTTP</code> и <code>HTTPS</code> , <code>NodePort 30080/30443</code>) → <code>HTTPRoute</code> → <code>Service</code> приложения.	<code>Envoy Gateway</code> — проект <code>CNCF</code> , изначально созданный под <code>Gateway API</code> . Не требует замены <code>CNI</code> или <code>service mesh</code> .	<code>kubectl get gateway,httproute -A</code> → Programmed, Accepted
04 Мониторинг	<code>kube-prometheus-stack</code> . <code>PodMonitor</code> и <code>ServiceMonitor</code> для <code>nginx</code> , <code>Envoy</code> , <code>Fluentd</code> и <code>VictoriaLogs</code> . Метрики <code>control plane</code> открыты в конфиге <code>kubeadm</code> .	Цели описаны декларативно рядом с каждым компонентом. <code>Grafana</code> и <code>Alertmanager</code> входят в стек.	запросы <code>PromQL</code> из <code>README §6</code> → все цели в состоянии <code>UP</code>
05 Логирование	<code>Fluentd DaemonSet</code> читает <code>/var/log/containers</code> , добавляет метаданные <code>Kubernetes</code> , разбирает <code>JSON</code> , разделяет <code>access</code> и <code>error</code> , пишет в <code>VictoriaLogs</code> .	Официальный образ <code>Fluentd</code> без доработок. <code>VictoriaLogs</code> занимает ~100 MB памяти, <code>Elasticsearch</code> — 1–2 GB.	запрос с <code>X-Request-ID</code> , затем поиск из <code>README §7</code> → запись в логах <code>nginx</code> и <code>Envoy</code>
06 Ubuntu 24.04	Скрипт проверяет ОС. Полный сценарий выполнен на чистой <code>VM Ubuntu 24.04</code> и выполняется в <code>CI</code> на каждом коммите.	Работа на целевой ОС подтверждается при каждом изменении.	<code>GitHub Actions</code> → задача <code>e2e</code>
07 Автоматизация	<code>make up</code> или три скрипта. <code>helm upgrade --install</code> и <code>kubectl apply --server-side</code> . Пароли генерируются при установке и хранятся в <code>Secret</code> .	Эксперту нужны только <code>bash</code> , <code>curl</code> и <code>git</code> . Повторный запуск ничего не ломает.	повторный <code>make up</code> → снова 42 из 42
08 Документация	<code>README</code> со всеми обязательными разделами; обоснование решений — <code>docs/DECISIONS.md</code> .	Любую проверку можно повторить копированием команды.	<code>README.md</code>

2 Реализованный функционал — дополнительные улучшения

Улучшения тоже покрыты автоматическими проверками и описаны в README §8.

улучшение	что и как реализовано	почему	как проверить
01 Маршрутизация Gateway API	Маршруты по пути (/v2), заголовку (X-Version) и хосту (v2.demo.local). Canary 80/20. HTTPS с сертификатом cert-manager и редирект с HTTP.	Реальные сценарии: canary-релиз, версии API, общий Gateway для нескольких команд.	команды из README §5
02 Политики Gateway	Retry и circuit breaker к приложению. Ограничение 10 запросов/с с ответом 429. Basic Auth на интерфейсы мониторинга и логов. Сквозной X-Request-ID .	Надёжность и защита на уровне шлюза, без изменений в приложении.	серия запросов → 429; UI без пароля → 401
03 Расширенный мониторинг	Запросы в секунду, коды ответов и задержки p50/p95/p99 на Gateway; CPU и память подов; 8 алертов в Alertmanager; дашборд Grafana с метриками и живыми логами.	Ключевые сигналы сервиса и контроль самой системы логирования.	<code>make traffic</code> , Grafana (<code>make creds</code>)
04 Логи с поиском	Поля status, uri, request_id, время ответа. Логи Gateway и приложения связаны одним request_id. Хранение 7 дней, свой интерфейс и Grafana.	Логи пригодны для расследований, а не только для чтения строк.	поиск по request_id (README §7)
05 CI	Проверка shell, YAML и Helm; проверка всех манифестов по схемам CRD. Затем на чистой Ubuntu 24.04: kubectl, развёртывание, 42 проверки, повторное развёртывание и проверки ещё раз.	Каждое изменение доказуемо работает и безопасно для повторного запуска.	GitHub Actions, <code>make lint</code>
06 Надёжность и безопасность	Не меньше 2 реплик, HPA, PDB, обновление без простоя; NetworkPolicy по умолчанию запрещает трафик; Pod Security restricted , non-root, файловая система только для чтения; секреты только в Secret.	Базовые практики production-кластера.	<code>kubectl get hpa,pdb,netpol -A</code>

3 Ревью работы и потенциальное масштабирование

Главная особенность

Решение доказуемо воспроизводимо: одна команда доводит чистую Ubuntu 24.04 до работающего кластера с Gateway API, метриками и логами, а CI на каждом коммите проходит этот путь дважды с 42 проверками. Наблюдаемость сквозная: один запрос с `X-Request-ID` находится в логах Envoy и nginx, а его код ответа и задержка видны в Prometheus.

Самое сложное решение

Выбор хранилища логов для Fluentd: Elasticsearch требует 1–2 GB памяти только под JVM, а для Loki нет плагина в официальном образе Fluentd — пришлось бы собирать свой образ. Выбран VictoriaLogs: около 100 MB памяти, приём JSON встроенным HTTP-выводом Fluentd, полнотекстовый поиск, свой интерфейс и плагин для Grafana.

Предложения по развитию

ЧТО ДОБАВИТЬ

ЧТО ДЛЯ ЭТОГО НУЖНО

Отказоустойчивый control plane из 3 узлов и публикация Gateway через MetalLB или BGP вместо NodePort

3+ VM, виртуальный IP; для BGP — пиринг с маршрутизаторами

GitOps (Argo CD или Flux): окружения dev → stage → prod из Git

доступ кластера к Git, отдельные кластеры окружений

ТЕЛЕКОМ
GRPCRoute, TCPRoute, UDPRoute для 5G SBA, SIP, Diameter и RADIUS

experimental channel Gateway API, тестовые CNF

ТЕЛЕКОМ
Ограничение запросов на абонента и вход через OIDC/JWT на Gateway

Redis для Envoy Rate Limit, корпоративный IdP

ТЕЛЕКОМ
Долгое хранение: VictoriaMetrics или Thanos, кластер VictoriaLogs, архив под требования регулятора

S3-совместимое хранилище (MinIO, Ceph), диски

Трейсинг: OpenTelemetry Collector и Tempo, трассы из Envoy

ресурсы под хранилище трасс

Безопасность поставки: сканирование Trivy, подпись cosign, политики Kyverno, Vault для секретов

Vault, приватный registry

ТЕЛЕКОМ
Установка без интернета на изолированных площадках edge и MEC

Harbor и зеркало Helm-чартов на площадке

ТЕЛЕКОМ
Нагрузки CNF: Multus, SR-IOV, DPDK, закрепление CPU и HugePages

серверы с SR-IOV-картами, настройка BIOS и ядра

Алерты в NOC: Telegram, почта, ServiceDesk; SLO-алерты по бюджету ошибок

токены каналов уведомлений через Secret

Известные ограничения. Один узел control plane; доступ через NodePort 30080/30443; данные на диске узла без репликации; сертификат от собственного CA для `demo.local`. Подробно — README §10.